Parsek - Timeline System Design

Comprehensive design specification for Parsek's unified timeline view - a chronological, read-only query layer across recordings and game actions that gives the player a single place to see everything that has happened and will happen on the committed timeline.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how the timeline aggregates and presents data from the flight recorder system (see `parsek-flight-recorder-design.md`) and the game actions system (see `parsek-game-actions-and-resources-recorder-design.md`).

Version: 1.0 (Implemented in v0.7) **Status:** Complete. Kept current through v0.10.5: the window layout in section 5 covers the Rewind/FF, Re-Fly and Career views (the Career view with its contract / strategy slot counts replaced the removed Career window), the six-button time range, Unfinished Flight Fly / Seal rows and the row hovers. **Out of scope:** Recording, playback, ghost visuals, vessel spawning. See `parsek-flight-recorder-design.md`. Resource tracking, ledger, recalculation engine. See `parsek-game-actions-and-resources-recorder-design.md`. Vessel-level telemetry (part events, segment events) - these remain in the Missions window's Recordings tab.

---

1. Introduction

Parsek has two existing views of committed data:

- The **Recordings tab** of the Missions window is vessel-centric: a list of recordings with per-recording controls (enable/disable, rewind, loop, group). It answers "what vessels did I record?"
- The **Game Actions window** (replaced by the timeline) was resource-centric: a flat list of ledger actions and legacy game state events with sorting and deletion. It answered "what resource transactions happened?"

Neither answered the question the player actually has: **what happened, and what will happen, in chronological order across all systems?**

The timeline is a unified, chronological view that pulls from recordings and game actions, normalizes them into a common entry shape, and presents them sorted by universal time (UT). It replaced the Game Actions window entirely.

1.1 What the timeline is

The timeline is a **read-only query layer**. It does not own data. Recordings remain in `RecordingStore`. Game actions remain in the `Ledger`. The timeline does not read those raw stores directly - it feeds from `EffectiveState.ComputeERS()` and `EffectiveState.ComputeELS()`, the re-fly supersede/tombstone-filtered effective state, so superseded recordings and tombstoned ledger actions are already excluded before the builder runs. From that effective state it constructs a flat list of entries and presents it.

This is analogous to a database view: it computes a result set from underlying tables. If a recording is committed or rewound, the timeline rebuilds. If a ledger action is added (facility upgrade, tech unlock), the timeline rebuilds. The timeline never modifies the data it reads.

1.2 What the timeline shows - and what it doesn't

The timeline shows **career-level events**: when missions launched and where vessels spawned, what milestones were achieved, what resources changed, when contracts completed, when the KSC was upgraded. These are the events that shape the player's career progression.

The timeline does **not** show vessel-level telemetry: engine ignitions, staging events, parachute deployments, solar panel toggles, RCS pulses. That detail belongs in the Recordings tab. A decoupler firing or an engine throttle change is not a career event - it's a flight detail.

The dividing line: **if it changed a resource, a contract, a milestone, a kerbal, or a facility - it's in the timeline. If it only changed a vessel's physical state - it's in the Recordings tab.**

1.3 What the player sees

The player sees the full committed history at all times. The timeline has a **current-UT marker** that divides the list into two visual halves:

- **Above the marker** (past): events that have already played out. Full color.
- **Below the marker** (future): events that will play out when the player advances time. Dimmed but not hidden.

There are no branches and no hidden future - the player recorded that future and committed it.

Situation	What the player sees in the timeline
Three recordings committed	All three recordings' career events interleaved chronologically - launches, spawns, milestones, science, contracts, all sorted by UT
Rewind to before a recording	That recording's events appear below the current-UT marker (future), dimmed.
Active flight in progress	Uncommitted events are not shown - they join the timeline on commit.
Facility upgrade at KSC	The upgrade action appears instantly in the timeline at the current UT.
Contract completed during a recording	The completion action appears at the UT it occurred, interleaved with other career events.

1.4 How the timeline differs from the Recordings tab

Aspect	Recordings tab	Timeline
Organized by	Vessel (one row per recording)	Time (one row per event)
Shows	Recording metadata, playback controls, chain/loop config, part events	Career events from all systems, interleaved chronologically
Controls	Enable/disable, rewind / fast-forward, loop, group, archive, re-fly	Rewind/FF, re-fly, watch, filters, warp to time, GoTo cross-link
Answers	"What recordings do I have? What did this vessel do?"	"What happened at UT 5000? How did my career progress?"

Both windows can be open simultaneously. The GoTo button on timeline entries opens the Missions window on the recording's mission (section 5.4).

Recordings tab grouping - a folder per mission. In the Recordings tab a group (folder) is the UI abstraction for one mission, i.e. one launch / one committed tree. Every committed tree gets its own auto-generated folder, **including single-recording launches** - a lone launch is still its own mission and renders as its own folder, not a bare root row. Folder names come from the vessel name and are de-duplicated with a `#N` suffix (`RecordingGroupStore.GenerateUniqueGroupName`), so flying the same craft twice produces two separate folders (`GDLV3` , `GDLV3 #2`) instead of merging both launches into one. Debris and EVA crew of a mission nest in `.../ Debris` and `.../ Crew` subfolders under their mission's folder. Auto-grouping runs at commit (`RecordingGroupStore.AutoGroupTreeRecordings`); the orphan-adoption helper only folds genuinely tree-less, time-overlapping split-segments into a mission folder and never adopts a separate prior mission that merely reused the same vessel id.

1.5 Data sources

1. Recordings (`EffectiveState.ComputeERS()` over `RecordingStore.CommittedRecordings`)

Each committed recording contributes:

- A **launch event** at `rec.StartUT` with vessel name and MET duration. EVA recordings show `EVA: Kerbal from Vessel (MET 5s)` .
- A **spawn event** at `rec.EndUT` with vessel situation context: `Spawn: Vessel (Landed on Mun)` . Boarded EVA kerbals show `Board: Kerbal (Vessel)` .
- A **crew death entry** per dead kerbal (bug #229) at `rec.EndUT` , read from `Recording.CrewEndStates` : `Lost: Bob Kerman (Vessel)` .
- A **separation entry** at the staging split point. A re-flyable slot emits an `UnfinishedFlightSeparation` row (`Unfinished Flight: Booster`) that carries Fly/Seal buttons so the player can re-fly the sibling or close the slot directly from the timeline; a post-merge or non-re-flyable split emits a plain `Separation` row (`Separation: Booster`) with only GoTo.
- Debris recordings and hidden recordings are skipped.

2. Game actions (`EffectiveState.ComputeELS()` over `Ledger.Actions`)

Each ledger action is a single timeline entry. Display text is humanized: science subjects split and spaced (`Crew Report @ Kerbin Launchpad`), tech nodes capitalized (`Basic Rocketry`), milestones with spaces (`First Launch`), strategy names from lookup table (`Aggressive Negotiations`). Crew assignments include vessel name (`Assign: Jeb (Pilot) on Mun Lander`). EVA self-assignments (kerbal assigned to own EVA vessel) are filtered out.

Game actions are classified as either **Actions** (deliberate player choices: tech unlock, build, hire, contract accept/cancel, facility upgrade/repair, strategies) or **Events** (gameplay consequences: milestones, science earned, contract complete/fail, reputation changes, crew

assignment/rescue).

3. Legacy events (`MilestoneStore.Milestones`)

Committed `GameStateEvent` entries in `MilestoneStore` that no ledger row duplicates are shown here (`TimelineBuilder.CollectLegacyEntries`); no load-time migration converts them into ledger rows. Uncommitted events are **not** shown.

---

2. Design Philosophy

2.1 Read-only, not read-write

The timeline displays data. It never modifies it. Rewind and fast-forward are operations on the game state (loading a quicksave, advancing UT), not on the timeline's data.

2.2 Time-centric, not vessel-centric or resource-centric

The timeline groups by **time**. This reveals causal chains:

> UT 600: Milestone: First Orbit +5000 funds UT 800: Upgrade Launch Pad -> Lv.2 -18000 UT 900: Complete: Orbit Kerbin +8000 funds

The player sees that the milestone funded the runway upgrade.

2.3 Career events, not vessel telemetry

The timeline shows things that change career state. It does not show what a vessel did physically - that's the Recordings tab's job.

2.4 Full visibility, no hidden future

Both past and future events are always visible. Future events are dimmed but never hidden.

2.5 Significance filtering

Two tiers: **Overview** (mission structure) and **Details** (all transactions). Default is Overview.

2.6 The timeline replaces the Game Actions window

The timeline lives in `UI/TimelineWindowUI.cs` , replacing `UI/ActionsWindowUI.cs` . Same extracted-window pattern as the other UI classes: constructor takes `ParsekUI` parent, `DrawIfOpen(Rect)` entry point, `IsOpen` property, `ReleaseInputLock()` .

---

3. Architecture

3.1 Coupling to other systems

```
RecordingStore --+
                 +-- EffectiveState.ComputeERS() / ComputeELS() --reads--> TimelineBuilder <--reads-- MilestoneStore
Ledger ----------+     (supersede/tombstone-filtered effective state)                            (legacy saves only)
```

The builder never reads `RecordingStore.CommittedRecordings` / `Ledger.Actions` directly; it reads the ERS/ELS effective state, which drops superseded recordings and tombstoned actions. Side effects run through the row and footer controls: rewind/FF delegate to `RecordingsTableUI.ShowRewindConfirmation()` / `ShowFastForwardConfirmation()` ; the Watch button enters/exits watch mode via `WatchModeController` ; Fly/Seal on Unfinished-Flight rows route through `RewindInvoker` ; and the Warp-to-time row drives `WarpToTimeController` . The builder itself never mutates the data it reads.

3.2 TimelineEntry - the normalized event shape

```
TimelineEntry
    double           UT             - universal time
```

```
    TimelineEntryType Type            - discriminator enum (see section 3.3)
    string            DisplayText     - one-line human-readable description
    TimelineSource    Source          - Recording, GameAction, or Legacy
    SignificanceTier  Tier            - T1 or T2
    Color             DisplayColor    - green (earning), red (spending), light blue (player action), white (neutral)
    string            RecordingId     - source recording ID; null for KSC-only actions
    string            VesselName      - vessel name if applicable; null otherwise
    bool              IsEffective     - false if zeroed by recalculation (duplicate milestone, etc.)
    bool              IsPlayerAction  - true = deliberate KSC action, false = gameplay event
```

`TimelineEntry` is a **view object** - constructed on demand, never serialized, never persisted.

3.3 TimelineEntryType

Recording lifecycle (5 types): `RecordingStart`, `VesselSpawn`, `CrewDeath`, `UnfinishedFlightSeparation`, `Separation`

Game actions (23 entry types; every live `GameActionType` member renders as a row; the nine reserved supply-route ids never reach the ledger since 0.10.6, when route events moved to the route event log, so the timeline has no route entries). Four later action types reuse an existing bucket instead of adding an entry type: `StrategyScienceDebit` / `StrategyScienceCredit` render as `ScienceSpending` / `ScienceEarning`, and `KerbalRecovered` (`Recovered: <name>`) and `KerbalExperience` (`XP: <name> (<career-log entries>)`, e.g. `XP: Jebediah Kerman (Landed Kerbin, Flight Kerbin, Recovered)`) render in the `KerbalAssignment` bucket. The entry types: `ScienceEarning`, `ScienceSpending`, `FundsEarning`, `FundsSpending`, `ReputationEarning`, `ReputationPenalty`, `MilestoneAchievement`, `ContractAccept`, `ContractComplete`, `ContractFail`, `ContractCancel`, `KerbalAssignment`, `KerbalHire`, `KerbalRescue`, `KerbalStandIn`, `FacilityUpgrade`, `FacilityDestruction`, `FacilityRepair`, `StrategyActivate`, `StrategyDeactivate`, `FundsInitial`, `ScienceInitial`, `ReputationInitial`

Legacy (1 type): `LegacyEvent`

Total: 29 types.

3.4 Entry collection

`TimelineBuilder.Build()` accepts data sources as parameters for testability. The window passes the ERS/ELS effective state (see 3.1) for the first two arguments:

```
TimelineBuilder.Build(
    IReadOnlyList<Recording> committedRecordings,   // EffectiveState.ComputeERS()
    IReadOnlyList<GameAction> ledgerActions,         // EffectiveState.ComputeELS()
    IReadOnlyList<Milestone> milestones,
    Func<GameStateEvent, bool> isLegacyEventVisible,
    Game.Modes? currentMode = null
) -> List<TimelineEntry>
```

`currentMode` drives mode-based visibility: the initial funds and reputation seeds are hidden in sandbox and mission modes, and in science mode only the science seed shows. Three collectors run, then the merged list is stable-sorted by UT and post-processed: `CompactAdjacentMilestoneEntries` merges same-milestone rows within a 0.1s tolerance, and de-dup passes drop EVA-branch crew-reshuffle actions (bug #228) and legacy events that already appear as ledger actions.

Recording Collector - emits `RecordingStart` (with MET duration, EVA detection, parent vessel resolution) and `VesselSpawn` at EndUT (with terminal state and VesselSituation), plus a `CrewDeath` row per dead kerbal (bug #229) and a `UnfinishedFlightSeparation` / `Separation` row at each staging split point. Skips archived (`Recording.Hidden`) and debris recordings, always: archive is managed only in the Recordings tab. Chain recordings show full chain duration. EVA detection via `EvaCrewName` or single-crew vessel name match.

Game Action Collector - maps every action into its bucket, humanizes display text (science subjects, tech nodes, milestones, strategies, crew assignments with vessel name), classifies as Action or Event via `IsPlayerAction`, demotes ineffective T1 entries to T2, resolves vessel name from RecordingId.

Legacy Collector - iterates committed milestones, keeps only events visible to the current timeline (untagged rows plus tagged rows whose recording id is in the committed/pending/active current branch), skips filtered event types, all entries at T2.

3.5 Cache invalidation

Rebuilt on: recording commit, rewind, KSC spending, scene change, warp exit. Current-UT divider updates every draw call via `Planetarium.GetUniversalTime()`.

3.6 Display text humanization

- **Science subjects**: `crewReport@KerbinSrfLaunchpad` -> `Crew Report @ Kerbin Launchpad` (camelCase split, `Srf Landed` / `Srf Splashed` /standalone `Srf` stripped)
- **Tech nodes**: `basicRocketry` -> `Basic Rocketry`
- **Milestones**: `FirstLaunch` -> `First Launch`, `/` -> `-` (body separator)
- **Strategies**: lookup table for 7 stock strategies (`AggressiveNeg` -> `Aggressive Negotiations`), camelCase fallback for mods
- **Duration**: `FormatDuration` with KSP calendar (6h days, 426d years), only non-zero components

---

4. Significance Tiers

4.1 T1 - Overview (default)

Entry Type	Why T1
RecordingStart	Mission launch - fundamental timeline anchor
VesselSpawn	Vessel materialization with terminal state - the outcome
CrewDeath	Crew fatality (bug #229) - roster loss; permanent, or until the stock crew respawn stamped at the death ("respawns after ...", owner ruling S8)
UnfinishedFlightSeparation	Re-flyable staging split - carries Fly/Seal buttons
MilestoneAchievement	One-time progression gate, often with large rewards
ContractComplete	Mission objective achieved
ContractFail	Mission objective failed
FacilityUpgrade	KSC progression
FacilityDestruction	KSC regression
FundsInitial	Career start - baseline reference
ScienceInitial	Career start
ReputationInitial	Career start

4.2 T2 - Detail

All remaining entry types: resource transactions (science/funds/rep earning and spending), contract accept/cancel, kerbal hire, crew assignment/rescue/stand-in, plain (post-merge or non-re-flyable) `Separation` rows, facility repair, strategy activate/deactivate, legacy events.

4.3 Visibility rules

- **Overview** (default): T1 only.
- **Details**: T1 + T2.
- **Ineffective entries**: demoted one tier (T1 -> T2). Rendered grey.

---

5. Player Interaction

The measured layout of the window - every button width, hover and empty state - is `docs/dev/design-gui-inventory.md` section 3.3. This section is the player-level contract.

5.1 Window layout

`Parsek - Timeline`, opened from the main window's `Timeline` button (kept in both UI complexity modes; nothing inside the window is mode-gated except that `GoTo` is gated by its target's key, which is visible in both modes). Zones, top to bottom:

Zone 1: Filter area - two or three rows on one six-cell grid; every button has the cell's width and every row is left-aligned, so a row of fewer than six buttons leaves empty cells on the right. There is no archive control: recordings archived in the Recordings tab never appear on the Timeline (section 3).

- Row 1: the one-at-a-time view group - `Overview` / `Details`, the two action views (`Rewind/FF`, `Re-Fly`) that restrict the list to rows carrying those buttons, and `Career`.
- Row 2, the selected view's context row, drawn only when the view has one: the source toggles (`Recordings` / `Actions` / `Events`) under Overview / Details; nothing under Rewind/FF and Re-Fly, where the time-range row moves up under row 1 instead of leaving an empty line; the career categories (`Contracts` / `Strategies` / `Facilities` / `Milestones` / `Tech`, single-select, the last one remembered) under Career. A category view shows every row of that category from both tiers, past and future, and ignores the source toggles. Categories come from the ledger action type (section 3), so Tech is tech unlocks only and never a strategy's science leg. The Career button and the category set follow the game mode: all five in Career, Facilities / Milestones / Tech in Science, no Career button in Sandbox. In Career mode the `Contracts` and `Strategies` hovers end with the slot counts (`Contract slots: 4 of 7 free now (2 active, 1 reserved for later).`).
- Last row, the time range, always drawn: `Last Day` / `Last 7d` / `Last 30d` / `This Year` / `All` / `Custom`, exactly one lit (`All` by default), so the range in force is always on screen. `Custom` shows the From / To sliders for an arbitrary UT range; a slider drag lights `Custom`, a preset turns `Custom` off and hides the sliders, and turning `Custom` off returns to `All`. The range applies in every view, category views included: a Last N range ends at the current time, so it hides future rows. The same range filters the Missions window's Recordings tab.

Zone 2: Entry List - a flat chronological list (no header row, no sort controls) with a `-- <date> (now)` divider at the current UT. Each row is a 160 px time label, the description and right-aligned action buttons. Past entries in full colour (green = earnings, red = penalties, light blue = player actions, white = recordings); future entries dimmed; an ineffective entry grey. In flight, each RecordingStart entry shows a `W` (Watch) button; every RecordingStart entry shows `R` (past) or `FF` (future) plus `GoTo`. UnfinishedFlightSeparation rows carry `Fly` / `Seal` plus `GoTo`; plain Separation rows carry only `GoTo`; CrewDeath rows have no action buttons. The hovered row's details (the not-counted reason, the stock control a future row holds, the row kind's terms) show in the help strip. Empty state: `No timeline entries.`

Zone 3: Warp-to-time row - a `Warp to time` button and Year / Day / Hour / Minute fields: it jumps the game clock to the entered date (rewinding first when the target is in the past) via `WarpToTimeController`, after a `Confirm: Warp to Time` dialog. In flight the warp is deferred to the next Space Center arrival so the scene-exit merge dialog handles the live recording first.

Zone 4: Footer - the single-line help strip, then `Close`.

5.2 Entry display examples

```
Launch: Kerbal X (MET 6m, 30s)                    - regular recording
EVA: Jeb from Kerbal X (MET 5s)                    - EVA recording
Spawn: Kerbal X (Landed on Mun)                    - vessel materializes at EndUT
Board: Jeb (Kerbal X)                              - EVA kerbal reboarded
Milestone: First Launch +5000 funds +2.5 rep        - humanized milestone
Complete: Orbit Kerbin +8000 funds                  - contract
Tech: Basic Rocketry -5.0 sci                       - humanized tech node
Crew Report @ Kerbin Launchpad +5.0 sci             - humanized science
Build -5000                                         - vessel build cost
Assign: Jeb (Pilot) on Kerbal X                    - crew with vessel
Activate: Aggressive Negotiations (25% Funds->Rep)  - full strategy name
Starting funds: 25000                               - career seed
```

A contract fail row reads `Expired: <name>` when it lands at or after the deadline of the accept it closes, else `Fail: <name>`. A facility destruction or repair names the facility, and one event's per-building rows within 1 s fold into one row.

5.3 Row actions

RecordingStart entries show `R` (past recordings with a rewind save) or `FF` (future recordings); both delegate to `RecordingsTableUI` methods - the same confirmation dialogs and execution as the Recordings tab. In flight they also show a `W` (Watch) button that enters or exits watch mode on the recording's live ghost through `WatchModeController` (`EnterWatchMode` / `ExitWatchMode`). A greyed `R` / `FF` carries the refusal as its hover.

UnfinishedFlightSeparation entries show `Fly` and `Seal` buttons. `Fly` opens the re-fly dialog (`RewindInvoker.ShowDialog`) on the sibling slot; `Seal` closes the slot. Both resolve the rewind-point slot through the same `RecordingsTableUI.ResolveUnfinishedFlightRewindRoute` path the Recordings tab uses.

5.4 GoTo cross-link

`GoTo` on RecordingStart and separation entries opens the Missions window on its Missions tab and scrolls to the recording's original mission (`RecordingsTableUI.ShowMissionForRecording` -> `MissionsWindowUI.RevealMissionForRecording`), expanding the mission if it is collapsed. Its hover is `Show this recording's mission`; a recording that belongs to no mission greys it with `This recording is not part of a mission`.

5.5 Ineffective entries

Demoted one tier (T1 -> T2). Rendered grey. The row's hover says why it was not counted.

---

6. Game Mode Considerations

- **Career**: every row type and all five Career categories.
- **Science mode**: fund / reputation / contract / strategy entries never appear; the Career view offers Facilities / Milestones / Tech.
- **Sandbox**: no Career view; only recording lifecycle events are shown.

---

7. What the Timeline Does Not Do

- **Vessel-level telemetry**: part events, segment events, flag events - Recordings tab only.
- **Resource sparkline / graphs**: deferred.
- **Individual event deletion**: read-only.
- **Sorting by columns**: always UT order.
- **Live UT marker during warp**: jumps on warp exit.
- **Uncommitted events**: not shown.